

QKF-Certifier: finite quotient kernels and width-independent KnownBits proofs

Leonid Shcherbakov

12 September 2026

Preprint, version 1.0

Abstract

QKF-Certifier checks source-bound certificates for a fragment of KnownBits abstract transformers. Its public release parses a restricted straight-line transfer language, replays local normalization steps, and checks the resulting coordinatewise mask function on nine one-bit abstract inputs. We prove that this finite check is necessary and sufficient for soundness at every positive width within the accepted fragment; equality of the nine output rows likewise characterizes optimality. The normalizer is separated from the replay checker, and a decreasing syntactic measure establishes termination of the release’s rewrite system.

The construction is placed within the finite quotient framework of Papers I and II. For finite ground specifications of a separately compatible binary operation, we give a sound, complete relation-matrix closure with explicit quotient feedback, and a coverage theorem for sparse semilattice representations. These algebraic kernels have their own certificate contract and are distinguished from the public word-program checker. For the latter, the accepted coordinatewise language has exactly 2^{18} raw-mask semantic classes, 3^9 everywhere-valid classes, and respectively 64, 64, and 16 sound classes for AND, OR, and XOR. The sound classes form Boolean lattices under accumulation of correct precision facts; additional monotonicity reduces the first two counts to 26.

The paper specifies the source semantics, all 29 rewrite-rule identifiers, trusted components, and the correspondence between claims, code, and tests. It is tied to QKF-Certifier v0.1.0a1 at an immutable commit. The supplement supplies reproducible finite checks and a version manifest so that later software and manuscript revisions can be compared without silently changing the scope of an earlier certificate.

Keywords. KnownBits; abstract transformers; finite quotients; congruence closure; proof certificates; term rewriting; width-independent verification.

1 A reference paper for the implemented certifier

An abstract transformer describes the possible results of a concrete operation from partial information about its inputs. KnownBits records which bits must be zero and which must be one. A candidate transformer can contain arithmetic, counts, shifts, helper calls, and conditional expressions even when its final behavior is a simple coordinatewise function. The problem addressed here is to recover that behavior by checked transformations and discharge the remaining semantic obligation on a complete finite interface.

The public implementation is *QKF-Certifier* [3]. Its first release certifies the AND, OR, and XOR transformers extracted from the NiceToMeetYou artifact [4, 5]. NiceToMeetYou synthesizes abstract transformers by combining sound components using meet. QKF-Certifier addresses the certification of a restricted resulting program under an explicit word-semantics contract. For an accepted coordinatewise residual, the final proof covers every positive bit

width and requires no separate per-width SMT obligation. Unsupported residual operations return an unresolved result.

This is Paper III of a three-paper series. Paper I [1] characterizes the quotients, forced values, and completions determined by incomplete action specifications. Paper II [2] identifies when ground consequences become visible and supplies the query-subterm and threshold-model theorems. Here these results support an exact finite algebraic kernel, while a separate semantic quotient of word expressions supports the implemented certifier. The two constructions are connected by explicit evaluation and kernel arguments, not by assuming that every transformer preserves the algebraic operations of Paper I.

Contributions. First, we specify a relation-matrix algorithm for a finite ground, separately compatible binary operation and prove its soundness, completeness, and termination. A sparse semilattice coverage theorem identifies every compatibility case that an implementation must preserve. Second, we characterize the complete finite semantics of the accepted coordinatewise KnownBits language, including its sound precision classes. Third, we prove the source-to-verdict composition theorem for the implemented normalization and replay architecture, enumerate the trusted rule families, and give a concrete termination measure. Finally, we bind the claims and experiments to a software commit and provide an explicit protocol for subsequent revisions.

Three computational objects. The following distinction governs both the proofs and the implementation claims.

Object	Equality or obligation represented	Implementation in this release
Ground partial core	Consequences of a finite ground presentation on represented terms	Research reference in the supplement.
Relation-QKF	Exact equalities between carrier and slice labels, including quotient feedback	Research reference; sparse C++ experiments are separate.
QKF-Certifier	Source-to-expression equality, then soundness of an externally specified word target	Public Python package v0.1.0a1.

The public package does not run Relation-QKF or compute minimum proof-depth horizons. In particular, the congruence-depth optimum from Paper II is not a property of its normalizer. A checked rewrite sequence establishes equivalence regardless of whether it has minimum length or depth.

Version addressed. The executable claims in this manuscript refer to tag `v0.1.0a1`, commit

`f93561682319e8b911ed32c01583f2a496dc59fc`.

The repository is <https://github.com/Len989/QKF-Certifier>. Its certificate format is `qkf-rewrite-v3`; the semantic contract begins `total-transfer-words-v2`. Section 9 explains how these identifiers evolve independently of the paper version. The full manifest is included in the supplement. A later branch head is not implicitly covered by the measurements or code audit reported here.

2 The finite partial core inherited from I and II

Let Σ be a finite finitary signature, E a finite set of ground equations, and $\mathcal{Q}$ a finite set of query terms. Constants have depth zero. Put

$$\begin{aligned} S &= \text{Sub}(E) \cup \text{Sub}(\mathcal{Q}), & S_D &= \{u \in S : \text{dep } u \leq D\}, \\ E_D &= \{s = t \in E : \max(\text{dep } s, \text{dep } t) \leq D\}. \end{aligned}$$

Let $\approx_D^S$ be congruence closure on the represented applications in S_D . Write $\equiv_D$ for the bounded *congruence-proof* equality of Paper II and $\equiv_E$ for full ground equality.

Proposition 2.1 (Partial core and evaluation, K1). *The quotient $\mathbf{P}_D = S_D / \approx_D^S$ is a well-defined partial algebra: a tuple has an operation value when it is represented by an application node, and its value is the parent class. Every total $M \models E_D$ induces an operation-preserving map*

$$\eta_M : \mathbf{P}_D \rightarrow M, \quad [u] \mapsto \llbracket u \rrbracket_M.$$

For active query terms s, t , their classes coincide exactly when $s \equiv_D t$. At $H = \max\{h(E), d_{\mathcal{Q}}\}$ this is equivalent to $E \vdash s = t$.

Proof. Congruence closure identifies the parents of any two represented tuples with componentwise equal arguments, so the partial operations are well defined. Every generating equation holds in M , and equality is preserved by its operations; hence evaluation factors through the quotient and preserves represented operations. The equality assertions are Paper II, Theorems 4.1 and 3.1, respectively. Empty query depth and empty axiom depth are taken to be zero. $\square$

This core precedes any choice of values for unrepresented tuples. A finite separating total model is obtained by assigning a default value to each unrepresented operation tuple and adding a default element to every empty active sort. That totalization is a certificate-producing construction, not additional canonical information in $\mathbf{P}_D$.

A positive derivation shows that two classes must merge. Conversely, a total model of E_D separating their labels shows that they cannot merge at that horizon. Paper II, Proposition 4.3 and Corollary 4.5, combine a chronological proof DAG at the first complete threshold with such a lower model. Their resource λ_E bounds all terms in a congruence proof. The different sequential resource λ_E^{rw} bounds whole intermediate rewrite terms. Neither is asserted to equal the number of normalization steps in the public certifier.

2.1 The same action example in all three papers

For the carrier $A = \{0, 1, 2, 3\}$, use the operation table

f	0	1	2	3
0	0	2	0	0
1	1	2	1	3
2	0	2	2	0
3	3	0	3	0

and one operator b , with $r_i = \alpha(b, i)$ and observations $r_0 = 0, r_2 = 2, r_3 = 0$. Include the native diagram and named-input compatibility. Paper I, Example 6.2, and Paper II, Section 5.1, give

$$\begin{aligned} \mathcal{I}/\equiv_1 &= \{\{0, r_0, r_3\}, \{1\}, \{2, r_2\}, \{3\}, \{r_1\}\}, \\ \mathcal{I}/\equiv_2 &= \{\{0, 2, r_0, r_1, r_2, r_3\}, \{1\}, \{3\}\}. \end{aligned}$$

For example, $f(0, 1) = 2$ and $f(3, 1) = 0$ yield $r_2 = f(r_0, r_1) = f(r_3, r_1) = r_0$. Likewise $f(1, 0) = 1$ and $f(1, 3) = 3$ yield $r_1 = r_3$. The canonical maps in Proposition 2.1 thus recover the five-to-three class change on exactly the named interface described in Paper I, Theorem 6.6.

This is an algebraic example, not a KnownBits program. Its role in III is to fix the meaning of a forced quotient and a finite interface before introducing specialized representations. Quotient-feedback rounds, syntactic depth, proof length, and semantic table size remain different quantities throughout.

3 Exact relation-matrix closure

3.1 Ground specification and labelled copies

Let A be a nonempty finite algebra, $|A| = n$, whose native operations have positive arity. Add a fresh binary symbol τ and observations $\Gamma \subseteq A^2 \times A$. The ground theory E_τ contains the complete named native diagram, the observed equations $\tau(a, b) = y$, and separate named-input compatibility. For each native k -ary g , include

$$\begin{aligned}\tau(g(a_1, \dots, a_k), b) &= g(\tau(a_1, b), \dots, \tau(a_k, b)), \\ \tau(b, g(a_1, \dots, a_k)) &= g(\tau(b, a_1), \dots, \tau(b, a_k)).\end{aligned}$$

All displayed arguments are carrier names. No universal identity for g on unnamed elements is imposed. The constants naming A are not nullary native operations required to be preserved by every slice.

Use copies $\Lambda = \{*\} \cup \{R_b : b \in A\} \cup \{C_a : a \in A\}$. A label (i, x) denotes

$$\ell_*(x) = x, \quad \ell_{R_b}(x) = \tau(x, b), \quad \ell_{C_a}(x) = \tau(a, x).$$

Two slice labels can represent one physical cell. Each relation $R_{ij} \subseteq A^2$ records proposed equalities between the labels in copies i and j .

3.2 The closure rules

Initialize diagonal reflexivity, physical-cell facts $(a, b) \in R_{R_b, C_a}$, and the facts $(a, y) \in R_{R_b, *}$ and $(b, y) \in R_{C_a, *}$ for every observation. Saturate the following rules:

- (R1) Converse: $(a, b) \in R_{ij}$ implies $(b, a) \in R_{ji}$.
- (R2) Composition: $(a, b) \in R_{ij}$ and $(b, c) \in R_{jk}$ imply $(a, c) \in R_{ik}$.
- (R3) Native compatibility: from $(a_l, b_l) \in R_{ij}$ for all $1 \leq l \leq k$, infer

$$(g^A(a_1, \dots, a_k), g^A(b_1, \dots, b_k)) \in R_{ij}.$$

- (R4) Local feedback: $R_{**} \subseteq R_{ii}$ for every copy i .
- (R5) Context feedback: $b R_{**} d$ implies $(x, x) \in R_{R_b, R_d}$ and $(x, x) \in R_{C_b, C_d}$ for every x .

Let R^* be the least closed matrix containing the initial facts.

Theorem 3.1 (Exact relation kernel, K2). *A fair saturation that returns only at a common fixed point terminates and computes R^* . For all interface labels,*

$$(x, y) \in R_{ij}^* \iff E_\tau \vdash \ell_i(x) = \ell_j(y).$$

*In particular $R_{**}^* = \theta_{E_\tau}^A$. The carrier values forced for $\tau(a, b)$ are exactly $\{y : (a, y) \in R_{R_b, *}^*\}$; a nonempty such set is one central-congruence class.*

Proof. There are $m = 1 + 2n$ copies and at most $m^2 n^2$ relation facts. Every productive rule application adds a fact. Finite monotone saturation therefore reaches the least fixed point; a worklist or complete-pass implementation must stop when no new fact is available.

For soundness, converse and composition express equality, and physical-cell facts identify the two names of the same term. Each copy preserves every named native operation by the diagram or compatibility equations. Thus native compatibility is sound between any pair of copies. Local and context feedback are congruence of τ in its varying and fixed arguments. All added facts are consequently ground consequences of E_τ .

For completeness, take any closed matrix R . On $V = \Lambda \times A$ put $(i, a) \sim (j, b)$ precisely when $(a, b) \in R_{ij}$. Reflexivity, converse, and composition make this an equivalence. Set $M = V/\sim$. On tuples represented in one copy, define

$$g^M([(i, a_1)], \dots, [(i, a_k)]) = [(i, g^A(a_1, \dots, a_k))].$$

If another representation of the same tuple uses copy j , native compatibility identifies the two proposed results. This defines a partial operation on classes; assign a fixed default class to all other tuples. Interpret carrier names by their central classes and put

$$\tau^M([(*, a)], [(*, b)]) = [(R_b, a)].$$

Local feedback makes the value independent of the representative of a ; context feedback does the same for b . Physical-cell facts give the equivalent column interpretation. Complete all remaining τ -tuples by a default class.

The named diagram, observations, and both compatibility families now hold by construction. Each interface label evaluates to its own class, so distinct R -classes are separated by a total model of E_τ . Applying this construction to R^* proves completeness. The assertion about forced values follows by transitivity with central equality. $\square$

The bound counts possible facts, not the running time of an inefficient repeated composition scan. For fixed native arities an explicit implementation is finite and polynomial in the explicitly represented carrier and rule-instance space; no polynomial bound in a succinct bit-width parameter follows. A native nullary operation would require additional zero-premise cross-copy facts and is excluded from this specification.

Reference algorithm. Initialize the labelled relations; repeatedly apply feedback, native compatibility, and full relational composition, inserting converse facts with each insertion. Return only when a complete pass makes no change. The supplement implements this deliberately transparent oracle and compares its entire carrier/cell partition with independent ground subterm closure. A finite resource limit must return unresolved, never an unfinished matrix presented as exact.

3.3 Certificates and two necessary closure mechanisms

Corollary 3.2 (Checking a proposed exact partition). *A proposed interface partition is exact if (i) generators of each of its blocks have checked derivations from the initial facts and rules, and (ii) the partition contains every initial fact and is closed under every rule.*

Proof. The derivations give $R \subseteq R^*$. Closure and the initial facts give $R^* \subseteq R$, by leastness. Equivalently, the first check excludes unjustified equalities and the second supplies the separating model from Theorem 3.1. $\square$

This algebraic certificate contract is separate from `qkf-rewrite-v3`, which certifies word-expression rewrites.

Example 3.3 (Composing generators is insufficient). In the one-bit KnownBits join-semilattice, write U for unknown and Z for known zero, so $U \vee_A Z = U$. Let $G = \{(U, U)\}$ and $H = \{(U, Z), (Z, U)\}$. The join-closure of H also contains (U, U) . Hence the composition of the two closed relations contains both (U, Z) and (U, U) . Composing only the displayed generators gives $\{(U, Z)\}$, whose join-closure still omits (U, U) . Rule R2 must quantify over the full represented relations, including derived intermediate witnesses.

Example 3.4 (Feedback determines additional cells). Take the two-element join-semilattice and supply both $\tau(0, 0) = 0$ and $\tau(0, 0) = 1$. The central carrier collapses. With feedback, all four named input cells become equal to that central class. If feedback is omitted while the other rules are saturated, only the observed cell is connected to the center. This finite example is checked in the supplement.

The same model proof extends to a fixed p -ary τ using $1 + pn^{p-1}$ copies, one for each varying coordinate and fixed context. All views of a physical cell must be identified, and feedback must cover every context coordinate. The implementation and differential results reported here concern the binary case.

4 Sparse semilattice representations

Suppose now $A = (A, \vee_A)$ is a finite join-semilattice and τ is binary. The relation matrix can be represented by global equality components of labelled endpoints instead of storing every R_{ij} explicitly. A missing physical cell still denotes a two-endpoint component, one in its row and one in its column. It is implicit, not absent.

For the current central quotient $Q = A/\theta$, materialize all central endpoints. Materializing either view of a cell also materializes its other view and identifies them. For each used copy i , maintain its exact local congruence $\kappa_i \in \text{Con}(Q)$ and materialize every affected nontrivial local class. This guarantees that an unmaterialized cell has not silently acquired further equalities.

Lemma 4.1 (Compatibility coverage). *After exact local closure, all binary native-compatibility obligations are covered by:*

- (i) *each pair of distinct explicit global components, compared in every copy in which both occur;*
- (ii) *each explicit component and implicit physical cell, compared across that cell's row and column when both occur in the explicit component.*

Repeated components and pairs of distinct implicit cells create no further cross-copy obligation.

Proof. Representatives of one component inside one copy are interchangeable modulo κ_i . This reduces two distinct explicit components to one pair of representatives per common copy. Repeating one component gives its own class by idempotence; local closure handles different representatives of that component. Commutativity allows unordered component pairs.

An implicit component occurs in exactly one row and one column. Thus its only possible nontrivial comparison with an explicit component is (ii). Two distinct implicit cells cannot share both their row and column, so they have at most one common copy and create no cross-copy identification. Repeating one implicit cell is again covered by idempotence. Local compatibility in a single copy is already included in exact local congruence closure. $\square$

Theorem 4.2 (Sparse refinement contract, K3). *An algorithm that preserves the representation above, fairly saturates the two cases of Lemma 4.1, and performs full central quotient feedback computes the same interface partition as Theorem 3.1, provided it reports success only after a common fixed point is reached.*

Proof. Every materialization, local merge, and component-compatibility merge is justified by a relation-matrix rule. When central conflicts occur, close them as a native congruence in A , replace the carrier by the resulting quotient, and restart from the projection of the original observations. These are sound central consequences and implement both local and context feedback.

At a fixed point without central conflicts, expand the implicit cells conceptually. The local congruences, Lemma 4.1, and the quotient representation satisfy every rule of Theorem 3.1. Soundness and leastness therefore identify the result with R^* . At a fixed Q , the number of possible labelled endpoints and equality merges is finite. The central congruence can change strictly at most $n - 1$ times. An uncapped fair monotone implementation consequently terminates; a prematurely exhausted engineering budget must return unresolved. $\square$

Algorithmic order. Start with the diagonal central congruence. Build Q and projected observations; close mirrors and local congruences, then both compatibility cases, repeating until stable or a central conflict is found. If a conflict appears, enlarge θ by its native congruence closure and rebuild over Q . Return only after a stable round with no conflict. This specification exposes the conditions an optimized representation must preserve, rather than assuming that a choice of data structure establishes completeness.

4.1 Principal generators and a width-dependent carrier

For $s \leq u$ in a join-semilattice, the comparably generated principal congruence satisfies

$$\text{Cg}(u, s) = \Delta \cup \{(a, b) : s \leq a, b, u \vee_A a = u \vee_A b\}. \quad (4.1)$$

We use this established characterization [8, Theorem 2.1], together with

$$\text{Cg}(x, y) = \text{Cg}(x \vee_A y, x) \vee \text{Cg}(x \vee_A y, y).$$

Grouping the elements $a \geq s$ by $u \vee_A a$ supplies generators for each class in (4.1). Inserting these pairs directly into a persistent local disjoint-set structure computes its join with the new principal congruence; rebuilding a separate full-carrier structure for each seed is unnecessary.

On nonempty KnownBits values at width w , native join retains the bits known in both arguments:

$$(Z, O) \vee_A (Z', O') = (Z \& Z', O \& O').$$

There are 3^w carrier values. Above a fixed element, the original discrete carrier consists of submasks of its known literals, so at most 2^w values need inspection for a comparable principal generator. After a nontrivial quotient, a general quotient-carrier scan remains a valid option. These observations concern representation cost in a width-dependent finite algebra; they do not give a polynomial-time algorithm in w .

The historical component-pair C++ backend implements these ideas and has substantial differential evidence. Its source includes a 100-pass loop without a distinct exhaustion result. Therefore Theorem 4.2 is a proved representation contract, not an unconditional correctness theorem for that exact historical file. The public Python certifier has its own explicit resource-limit outcomes and does not call this backend.

4.2 Algebraic compatibility does not establish concrete soundness

A constant τ preserves a positive-arity semilattice operation separately in both arguments. Choosing its constant to be a wrong concrete answer gives an algebraically compatible but

unsound transformer. Consequently relation closure cannot turn arbitrary sampled examples into a soundness proof for an unrestricted candidate language.

Nor does every optimal transformer preserve this native join. At width two, let $a = 00$, $a' = 11$, and $b = 01$ denote exact words. Then $a \vee_A a' = ??$, and the best modular-add transformer satisfies

$$\text{Add}^\#(a \vee_A a', b) = ??, \quad \text{Add}^\#(a, b) \vee_A \text{Add}^\#(a', b) = 01 \vee_A 00 = 0?.$$

Thus separate join compatibility is not a general axiom of optimal KnownBits addition. The word-certification route below uses the actual expression semantics instead of importing such an axiom.

5 The complete finite semantics of the accepted word fragment

5.1 KnownBits and the coordinatewise language

For $w \geq 1$, write $B_w = \{0, \dots, 2^w - 1\}$ and let

$$\text{KB}_w = \{(Z, O) \in B_w^2 : Z \& O = 0\}.$$

Its concretization is

$$\gamma_w(Z, O) = \{x \in B_w : x \& Z = 0, x \& O = O\}.$$

Inputs are nonbottom: every bit is known zero, known one, or unknown. We use $\mathbb{K} = \{Z, O, U\}$ for these three one-bit states, encoded respectively by $(1, 0)$, $(0, 1)$, and $(0, 0)$. The context distinguishes the state symbols from word masks. An output with conflicting masks has empty concretization and cannot be sound for the nonempty target images considered here.

The accepted mask language has four variables z_a, o_a, z_b, o_b , the constants zero and all-ones, and bitwise AND, OR, XOR, and NOT. A pair $e = (e_Z, e_O)$ consists of two such expressions. The same syntax is interpreted at every width. The ordinary integer constant 1 is not an all-ones constant at widths above one and is therefore outside this final language, as are counts, shifts, comparisons, and arithmetic residuals.

KnownBits and the closely related tnum domain are established abstract domains [6, 7]. We use raw zero/one masks so that the exact interface matches the program's return value. Our class counts below do not identify distinct invalid masks merely because they denote bottom.

Lemma 5.1 (Coordinate evaluation). *For every accepted mask expression e , positive width w , and bit position $j < w$, its output bit depends only on the four input-mask bits at j and equals the one-bit evaluation of e on those bits.*

Proof. Variables and the two constants have the stated property. Each permitted operation applies its Boolean operation independently at each bit. Structural induction gives the result. $\square$

Theorem 5.2 (Nine-row soundness and optimality, K5.1). *Let f_w be the coordinatewise lift of a Boolean binary target f_1 . An accepted pair expression e is sound at every positive width if and only if, for every $(a, b) \in \mathbb{K}^2$,*

$$\{f_1(x, y) : x \in \gamma_1(a), y \in \gamma_1(b)\} \subseteq \gamma_1(e_1(a, b)). \quad (5.1)$$

Replacing inclusion by equality gives a necessary and sufficient criterion for the best KnownBits result at all positive widths. A failed soundness row yields a concrete width-one counterexample.

Proof. Necessity follows by choosing width one. For sufficiency, every valid word input is a product of its one-bit possibilities. By Lemma 5.1, (5.1) at each coordinate includes the corresponding concrete target bit in the output. This proves soundness of the whole word.

For optimality, coordinatewise target evaluation on product inputs has a product image: choices of the input bits at different positions are independent. Each coordinate image is a nonempty subset of $\{0, 1\}$ and is represented exactly by one of Z, O, U . Equality in all rows therefore gives exactly the product image, which is the best domain value. If inclusion fails, the offending row contains concrete bits whose target result violates an output mask, already at width one. $\square$

For the three supported targets the exact table is:

a	b	AND	OR	XOR
Z	Z	Z	Z	Z
Z	O	Z	O	O
Z	U	Z	U	U
O	Z	Z	O	O
O	O	O	O	Z
O	U	U	O	U
U	Z	Z	U	U
U	O	U	O	U
U	U	U	U	U

5.2 An extensional quotient, not a syntactic normal form

Order the input rows lexicographically using (Z, O, U) on each argument, with the second argument varying fastest. Encode the zero and one output bits of row i in positions $2i$ and $2i + 1$:

$$\sigma(e) = \sum_{i=0}^8 \left(e_Z(i)2^{2i} + e_O(i)2^{2i+1} \right).$$

This is the exact 18-bit signature reported by `inspect`.

Theorem 5.3 (Complete finite signature, K5.2). *Two accepted pair expressions have equal signatures exactly when they return identical masks at every positive width on every valid input. There are exactly 2^{18} raw-mask semantic classes and 3^9 classes with valid outputs on every valid input. Evaluation of one mask is a Boolean-algebra homomorphism onto the 512-element algebra of Boolean functions on the nine rows.*

Proof. Equality of signatures is equality on the nine rows and lifts coordinatewise by Lemma 5.1; conversely, take width one. To show surjectivity, each valid assignment of the four mask bits has a Boolean minterm using variables and their negations that singles it out among the nine rows. Disjunctions of these minterms define every subset of rows. Thus one mask has 2^9 classes, and two independently chosen masks have 2^{18} . Restricting each row to the three nonconflicting outputs gives 3^9 . Boolean operations are evaluated pointwise, proving the homomorphism assertion. $\square$

The kernel of this evaluation is an exact semantic congruence. It is a second concrete instance of the evaluation-kernel viewpoint from Proposition 2.1, but its semantic models and permitted expressions are specified independently of E_τ . The implemented normalizer is not claimed to choose one unique syntax for each signature.

For componentwise mask meet, $(Z, O) \sqcap (Z', O') = (Z \mid Z', O \mid O')$, the signature obeys

$$\sigma(e \sqcap e') = \sigma(e) \text{ OR } \sigma(e'). \quad (5.2)$$

On arbitrary raw outputs this operation can produce conflicting masks. On sound transformers of the same target, the common nonempty concrete image prevents that conflict.

5.3 The exact precision classes

Let $\mathfrak{F}_f$ be the input rows on which the target image is a singleton, annotated by that forced bit. These are the possible correct precision facts. No monotonicity assumption is imposed in the following classification.

Theorem 5.4 (Sound precision lattice, K5.3). *Sound coordinatewise transformer classes for f are in bijection with subsets of $\mathfrak{F}_f$. Every subset is expressible in the accepted language. Meet corresponds to union of precision facts. For AND, OR, and XOR there are respectively 64, 64, and 16 sound classes, and a chain of strict meet refinements starting at all-unknown has at most 6, 6, and 4 strict steps.*

Proof. When the target row image is $\{0, 1\}$, soundness forces output U . When the image is a singleton, the only sound outputs are its correct singleton and U . Thus a sound table chooses exactly which forced facts to retain. Theorem 5.3 gives definability of every such table, and (5.2) gives union. The displayed target table has six, six, and four forced rows. Each strict refinement adds at least one previously absent fact. $\square$

These heights bound semantic precision within the accepted fragment. They do not bound candidate-generation iterations, proof-search cost, or the cost of the best implementation of a class.

Proposition 5.5 (Additional monotonicity). *If transformers must also be monotone for the precision order induced by concretization inclusion, the numbers of sound classes are 26, 26, and 16 for AND, OR, and XOR.*

Proof. A retained forced fact at an input row requires the same fact at every more precise row. For AND, the five zero rows have three minimal rows (Z, Z) , (Z, O) , (O, Z) and two further rows (Z, U) , (U, Z) . Selecting (Z, U) requires the first two minimal rows; selecting (U, Z) requires the first and third. The number of downsets is $8 + 2 + 2 + 1 = 13$, according to whether neither, only the first, only the second, or both further rows are selected. The one row (O, O) forcing one is independently optional, giving 26. OR is dual. All four forced XOR rows are minimal, so every subset is monotone. $\square$

The release computes the 18-bit signature and target soundness/optimalty; it does not expose a separate monotonicity flag. The additional classification is a mathematical result checked in the supplement and a possible future interface extension.

6 From source text to a checked all-width verdict

6.1 The source-semantics contract

The source bundle maps caller-chosen labels to UTF-8 program texts. The entry function takes two KnownBits pairs. The supported grammar is a restricted, line-oriented subset of transfer MLIR: single-block typed SSA functions, pure operations, acyclic calls with explicitly supplied definitions, and an actual return instruction. It is not the full MLIR language. Unknown

syntax, unresolved calls, recursion, type inconsistencies, and exhausted resource budgets do not produce a successful target verdict.

At width $w \geq 1$, word arithmetic is modulo 2^w . Boolean expressions have separate two-valued semantics. Counts return integers in $[0, w]$, encoded as words; $w < 2^w$ ensures this is possible. Shifts and division use the total semantics specified in Appendix A. In particular, the source operation `transfer.neg` denotes bitwise NOT in this contract. No poison or undefined behavior is introduced. Since every operation is pure and total, discarding an unused computation preserves the returned value.

Lemma 6.1 (Expression construction). *For this grammar and semantics, symbolic evaluation of SSA instructions, expansion of acyclic calls, and projection of the actual return produce an expression e_0 with the same value as the source entry at every positive width on every valid input.*

Proof. Maintain an environment mapping each defined SSA name to an expression with its current semantic value. Input projections initialize the four mask variables. Pair construction and projection preserve the pair semantics. Each primitive instruction adds its specified operation to already-correct operand expressions. A call substitutes the actual argument expressions for the callee’s formal arguments; induction on the acyclic call graph justifies its expansion. Induction on instruction order maintains the environment invariant. Selecting the expression for the actual return gives the result. Totality justifies ignoring expressions that do not contribute to that return. $\square$

The frontend module performs this translation through its `parse_bundle` and `expression` functions. The lemma describes the translation algorithm; correspondence of the Python parser with the documented text grammar is part of the implementation trust described below.

6.2 Local rules and semantic guards

The release has 29 rule identifiers. Most implement direct constant, idempotence, cancellation, count, comparison, and selection laws. Two guards reason about support: `disjoint` establishes that two operands have no common one bits, and `subset` establishes inclusion of one operand’s support in another’s. On coordinatewise expressions they exhaust the nine valid input rows; structural cases use AND restriction and OR inclusion. The `lowbit` guard proves a constant low bit using bitwise laws and parity of modular addition/subtraction.

Lemma 6.2 (Sound guards). *If the release’s support guards return true, their respective disjointness or inclusion assertions hold at every positive width on every valid input. If `lowbit` returns $c \in \{0, 1\}$, the operand has low bit c at every positive width.*

Proof. For coordinatewise guards, Lemma 5.1 lifts the exhaustive one-bit checks. The structural disjointness cases replace an AND-expression by an operand with a superset of its support, so established disjointness is preserved. The subset cases use reflexivity, empty support, full support, inclusion into either branch of OR, and inclusion from either operand of AND.

For the low bit, zero, all-ones, and integer literals have their stated parity. A shift left by the word constant one clears that bit, including at $w = 1$. NOT flips it, AND/OR use their Boolean laws, and XOR, modular addition, and modular subtraction have low bit equal to parity XOR. Each recursive case is therefore sound. $\square$

For example, if $x \& y = 0$, addition has no carries and $x + y = x | y$ at every width. If $\text{supp}(x) \subseteq \text{supp}(y)$, setting the leading `countl_one(x)` bits of y changes nothing. These are precisely the guarded rules `disjoint-add` and `set-leading-included`. Appendix A gives the complete rule catalogue and proof arguments for the remaining families.

Proposition 6.3 (Local rewrite preservation). *Every applicable rewrite in the mathematical rule system listed in Appendix A preserves value at every positive width on valid KnownBits inputs.*

Proof. The guarded rules use Lemma 6.2. The remaining rules follow from the stated modular, Boolean, order, count, and total division/shift semantics, as checked by the cases in the appendix. In particular, the rules involving zero remainders remain valid at divisor zero under this contract. Replacing an equal-valued subexpression preserves every enclosing operation by congruence. $\square$

6.3 Termination without a normal-form optimality claim

For an expression tree e , let $N(e)$ count operation and leaf nodes, with numeric literal payloads treated as part of a leaf, and let $A(e)$ count occurrences of word addition. Shared occurrences are counted as occurrences in the expanded tree. Put

$$\mu(e) = N(e) + A(e).$$

Proposition 6.4 (A decreasing measure). *Every rewrite step of the release decreases μ strictly. Thus any rewrite sequence has at most $\mu(e_0) - \mu(e_N)$ steps. In particular the unbudgeted normalization strategy terminates.*

Proof. The rule `disjoint-add` changes an addition node to an OR node without changing its children; it preserves N and decreases A by one. Every other rule replaces a term by a proper subterm or a single constant, hence strictly decreases N and introduces no addition. In an enclosing context the change in μ is unchanged. The nonnegative integer measure proves termination and the bound. $\square$

This statement does not claim confluence, a unique syntactic normal form, minimum proof length, or minimum depth. The producer uses a fixed rule order after recursively processing children. Its finite proof-step budget can stop a terminating computation early; the API then returns `fallback_required`. The extensional signature of Section 5, when applicable, provides canonical semantic information independently of syntactic choices.

6.4 Replay and the composition theorem

A certificate records the schema identifier, source-label fingerprints, entry, semantic contract, initial-expression fingerprint, a sequence of rule/path instructions, and a claimed final expression. For a target certificate it also records the nine rows and target name. The caller supplies the source bundle and target again. The checker reparses that source, checks the bindings, replays every instruction, compares the obtained expression with the claimed one, and recomputes the target rows. A certificate cannot introduce an unproved assumption or substitute its own target for the caller’s target.

Theorem 6.5 (Source-to-verdict composition, K4). *Assume the frontend implements the source contract of Lemma 6.1, the checker implements the rules of Proposition 6.3, and replay successfully establishes $e_0 \rightsquigarrow e_N$. Then the source and e_N return the same value at every positive width on valid inputs. If e_N is an accepted pair and its recomputed table for the external target passes, the source transformer is sound at every positive width. If every table row is optimal, so is the source transformer. A failed row supplies a width-one refutation of the source.*

Proof. Induct on the replay instructions. Each instruction selects a valid subtree path and replaces its current subexpression by an instance of a value-preserving rule, establishing equivalence of successive whole expressions. Transitivity and Lemma 6.1 establish source equivalence with e_N . Apply Theorem 5.2 to the recomputed rows. Its concrete witness also refutes the source because normalization preserved the returned masks. $\square$

The producer’s search strategy is absent from this implication. The implementation’s checker does not run that search, although producer and checker share local-rule code. The trusted implementation comprises parsing, call expansion, schema validation, guards, local rules, replay, and the finite table evaluator. The mathematical proofs and finite regressions have not been replaced by a proof-assistant verification of that Python code. Nor does this theorem certify a separate compiler lowering of the source language.

6.5 Result semantics

Result	Meaning under the declared contract
certified	Target soundness at all positive widths; optimal reports the separate precision check.
unsound	Replayed normalization and a concrete width-one target counterexample.
normalized	Source-to-expression equality only; no target soundness claim.
fallback_required	Unsupported source/target/residual or exhausted budget; no complete target verdict.
invalid_certificate	A supplied certificate fails validation, binding, or replay.

Input and internal errors are separate failure outcomes. A normalization-only certificate has a null target and no rows; a target check does not accept it as a soundness certificate. Consumers must accept a transformer only from a successful target verification or check with status **certified**.

7 The public implementation and its scientific interface

7.1 What the released tool computes

The pinned release exposes inspection, normalization, target verification, and certificate checking through a Python API and command-line interface. Runtime dependencies are the Python standard library. Verification first constructs the source expression, produces a local rewrite trace, and attempts the coordinatewise target check. Checking reconstructs the source expression and replays the supplied trace without invoking the normalizer. These paths have different responsibilities even though they share the primitive-rule implementation.

The certificate format is **qkf-rewrite-v3**; API results use **qkf-result-v1**. The source contract identifier is recorded literally as

```
total-transfer-words-v2; positive-width;
modular-arithmetic; SMT-total-div-shift;
standard-counts; nonbottom-KB
```

It is a contract identifier, not a claim about all implementations of transfer MLIR. Resource limits bound source size, parsed functions and operations, call expansion, expression size/depth, certificate size, and proof steps. These are operational limits; the all-width theorem does not depend on testing a largest word width.

The source-label map is fingerprinted using SHA-256 over each UTF-8 text. Helper definitions are included, and the actual helper return is expanded. Whitespace edits therefore require regenerating the source-bound certificate even when they preserve program meaning. Fingerprints identify artifacts; they are not digital signatures establishing authorship. The source expression, entry, contract, external target, trace, and recomputed result must all agree.

7.2 Three concrete programs

The release includes examples from the Nice to Meet You artifact, with upstream commit and MIT attribution preserved in `examples/ntmy/SOURCES.json` and NOTICE. For AND, two guarded leading-bit updates and an idempotent maximum collapse to

$$(z_a \mid z_b, o_a \& o_b).$$

For OR, the normalized result is extensionally

$$(z_a \& z_b, o_a \mid o_b).$$

The final syntax need not be the shortest spelling of that formula. The XOR program combines two components through the supplied meet helper. One component contributes

$$(o_a \& o_b, (o_b \& z_a) + (z_b \& o_a)),$$

and the other contributes $(z_a \& z_b, 0)$ after count and clearing simplifications. The two summands have disjoint support on valid inputs: overlap would require both z_a and o_a , and both z_b and o_b , at the same position. The guarded addition rule changes their sum to OR. Expanding the actual meet helper then gives the optimal XOR table. Changing that helper changes what is certified; the checker does not recognize a function name and assume its intended semantics.

7.3 A map from claims to code and evidence

Table 1 is the entry point for maintainers. The labels K1–K5 are stable claim identifiers across manuscript revisions; theorem numbers are local to this version. Source links resolve to the exact commit in Section 1.

The public tests are in `test_core.py`, `test_properties.py`, `test_mutations.py`, `test_frontend.py`, and `test_cli.py`. The supplement supplies a separate release audit, a finite-algebra reference implementation, generated certificates, raw result JSON, and a manifest of source hashes. It retrieves the public code by immutable commit instead of presenting a second evolving copy of the repository.

8 Reproducible evidence and its limits

8.1 Fresh checks for this manuscript

All results in this subsection were rerun for this manuscript against the exact commit identified in Section 1. The environment was Python 3.12.14 on x86-64 Linux. The release suite passed all 67 tests. It includes exhaustive evaluation of the three original programs on every valid input pair at widths one through four, totalling 22,140 program/input comparisons, as well as property-based, parser, binding, schema, and CLI tests.

The independent matrix/ground comparison passed 804 finite instances and 35,856 interface-pair comparisons. These comprise 592 cases from every two-element binary table and 37

Claim	Implementation location	Evidence and boundary
K1, finite core	Supplement’s independent ground-closure comparison	Proposition 2.1; inherited locality from II. Not a public API computation.
K2, matrix closure	Supplement: <code>relation_matrix_exact.py</code>	Theorem 3.1; 804 differential cases and final closure checks. Research reference implementation.
K3, sparse representation	Historical component C++ prototype	Lemma 4.1 and Theorem 4.2; conditional representation contract, not a theorem about every prototype execution.
K4, source/replay	<code>frontend.py</code> ; <code>certificate.py</code> ;	Theorem 6.5; parsing, binding, tampering, mutation, and API tests.
K5, finite semantics	<code>kernel.py</code> , replay/check	Theorems 5.2–5.4; exhaustive valid tables and concrete word oracle.
TERM, termination	<code>kernel.py</code> , coordinate/bit/table	Proposition 6.4; all 29 rule witnesses and measured example traces.
MONO, monotone classes	<code>apply_rule</code> ; <code>producer.py</code>	Proposition 5.5; exhaustive count. No release monotonicity flag.

Table 1: Scientific claims, implemented components, and limits.

observation sets, 148 from every two-element unary table and the same observation family, and 64 deterministic random three-element mixed-signature cases. Each final relation matrix was checked for closure. The largest observed iteration count was five. The common example from I–II and the central-feedback counterexample also passed. These finite comparisons exercise the implementation; the general exactness argument is Theorem 3.1.

The release audit constructs a witness for each of the 29 rewrite identifiers and compares its before/after values using a concrete word evaluator on all valid input pairs at widths one, two, and three. All 23,751 comparisons passed. Every witness decreases μ , as do all steps in the three source-derived traces. Witness tests do not exhaust every possible instance of a rule; Proposition 6.3 supplies the general argument.

The signature audit constructs all 512 Boolean row functions by disjunctive normal form and evaluates them with the release’s bit evaluator. It independently enumerates all $3^9 = 19,683$ valid output tables per target, confirming the sound-class counts 64, 64, 16 and the additional monotone counts 26, 26, 16. Every pair of sound tables was checked against the signature-OR meet law: 4,096 AND pairs, 4,096 OR pairs, and 256 XOR pairs. Counterexamples to generator-only relational composition and to separate join preservation of best addition were also checked.

8.2 Certificates and mutation outcomes

Table 2 reports actual generated certificates. A node count here counts structurally distinct expression nodes; it differs from the expanded-tree measure in Proposition 6.4. Certificate sizes use compact UTF-8 JSON serialization.

Target	Nodes before/after	Steps	Bytes	Optimal	Signature
AND	10/7	3	1,251	yes	01255
OR	21/8	8	1,465	yes	08a89
XOR	20/11	7	1,544	yes	00189

Table 2: Source-bound certificates for the three pinned examples.

The deterministic mutation family modifies actual source operations, constants, and related constructs. Of 680 distinct mutations, 270 were certified sound, 180 received a width-one refutation, and 230 required fallback. Broken or unsupported residual expressions are not counted as proofs of unsoundness. Table 3 separates the targets.

Target	Mutations	Certified	Unsound	Fallback
AND	85	13	34	38
OR	356	167	62	127
XOR	239	90	84	65
Total	680	270	180	230

Table 3: Fresh source-mutation classification.

Including the three originals, 453 candidates reach the coordinatewise fragment: 48 AND, 230 OR, and 175 XOR. Within each target, they yield respectively 20, 28, and 50 distinct signatures. Thus target-indexed semantic deduplication reduces these 453 candidates to 98 classes; this sum does not assert that signatures in different target groups are disjoint. The reduction is empirical for this mutation family, not a general bound on search improvement.

For a small reproducibility check, 31 warmed calls per example gave median end-to-end verification times of 3.58, 5.61, and 7.45 ms for AND, OR, and XOR; certificate checking took 1.58, 2.14, and 3.26 ms. Raw minimum, median, maximum, and environment fields are retained in the supplement. These are shared-host observations including parsing and validation, not controlled cross-system benchmarks or evidence of a speedup over Nice to Meet You.

8.3 Earlier research evidence, kept separate

Earlier project records report 3,720 baseline/component comparisons and 3,000 component/fast-local comparisons on small KnownBits carriers. A recorded width-eight run has $|\text{KB}_8| = 6,561$, 1,000 observations, 364,572 forced cells, and 831,389 materialized endpoints; its recorded time is 14.20 s with peak resident memory 449,444 KiB. These are historical measurements from the research prototype, not reruns of the public Python release. The supplement preserves the underlying small result records and their provenance. They motivate the sparse representation but do not remove its implementation obligations or its pass-limit issue.

An earlier transformer-corpus pass normalized all 39 inspected solutions but reached the coordinatewise fragment for only three. That historical coverage result is consistent with the present scope: arithmetic, shifts, and counts can disappear under sound local rules, but residual cross-bit dependence is outside the nine-row theorem. In particular, the public release does not supply a certified general addition transformer or a general substitute for the upstream solver pipeline.

9 Keeping the paper and software connected

This paper is the primary scientific reference for QKF-Certifier. Papers I and II remain the foundations for initial semantics and equality visibility; they should not need rewriting whenever a parser feature or rewrite rule is added. The repository should link to this paper from its description of the algorithm, while preserving a separate software citation for reproducibility.

Four version axes must remain explicit: the manuscript version, the software version and immutable commit, the certificate schema, and the semantic contract. In this version they are respectively manuscript 1.0, software 0.1.0a1 at the pinned commit, `qkf-rewrite-v3`, and the contract quoted in Section 7. A theorem about an abstract rule system and a test result

for a particular commit have different lifetimes. Neither a moving branch URL nor a paper’s title identifies the code that was checked.

The supplement’s `CODE_VERSION.json` records this tuple, source-file SHA-256 values, example provenance, and claim identifiers. A repository-facing version records the same tuple and points back to the paper. For a deposited paper, maintainers should add both its version-specific DOI, which fixes the cited text, and its concept DOI, which resolves to the evolving paper record. No DOI is invented in the present pre-deposit package. A later software release should cite the specific manuscript version that describes its semantics, while a general README link may lead to the latest paper version.

Change	Scientific work required	Version/evidence action
Normalizer strategy or performance	Preserve the rule set and check replay behavior; update claims only if behavior changes.	New software commit and measurements; retain old evidence.
New local rewrite rule	Prove the rule under the declared semantics; revisit guards and termination.	Update the rule catalogue, witnesses, and claim map.
Frontend operation or helper handling	Specify semantics and prove expression construction; test source binding and returns.	Update supported grammar and semantic contract if meaning changes.
New target or residual language	Supply the appropriate lifting/decision theorem; nine rows alone do not cover cross-bit behavior.	New target scope, evidence, and manuscript revision.
Certificate representation	Specify accepted fields, validation, and replay compatibility.	Advance schema when incompatible; do not reinterpret old certificates.
Relation backend integration	Meet K2/K3 closure and representation obligations; expose budget exhaustion as failure.	Add implementation mapping and independent differential evidence.

Table 4: A change protocol for the evolving scientific reference.

The package includes proposed `CITATION.cff`, a README citation block, a paper-link document, and a version manifest. They are reviewable repository additions, not a claim that the remote repository has already been changed. After actual deposition, reserved or issued identifiers can replace the explicit null DOI fields. Old manuscript packages and their manifests should remain available.

The next mathematical extensions have concrete obligations. A carry-state proof for addition must connect the finite transition invariant to source semantics at every width. A symbolic relation backend must establish exact compatible closure and detect unfinished saturation. A broader search interface may rank and deduplicate candidates by signatures only after it has established the relevant semantic contract. These extensions can be added to this paper without obscuring which claims support the current release.

10 Relation to previous work

Nice to Meet You synthesizes practical MLIR abstract transformers and combines components using abstract meet [4, 5]. Its paper describes candidate soundness checking with Z3 across widths one through 64, discarding candidates on timeout. Our work studies a complementary route for a restricted residual language: replay establishes source equivalence and a product argument makes nine rows decisive for every positive width. It does not reproduce that

system’s synthesis coverage or replace its general solver use. The meaning of source operations must be matched explicitly before comparing verdicts; this paper’s total word contract is not an implicit verification of the upstream compiler lowering.

Known-zero/known-one representations are established in LLVM [6]. The analysis of tristate numbers gives soundness and precision results for arithmetic transformers, including addition and subtraction [7]. We do not claim these abstract domains or their standard bitwise formulas as new. Here they provide a concrete finite semantic quotient, a complete classification of the specified coordinatewise fragment, and a target check attached to a source-bound certificate.

Bit-width-independent SMT encodings study a broader route through integer arithmetic, uninterpreted functions, and quantified constraints [9]. Our finite product argument is simpler because its residual expressions do not communicate between bit positions. It offers no general decision procedure for parametric bit-vector arithmetic. Alive2 instead checks LLVM transformations through bounded translation validation [10]; its compiler refinement setting and treatment of the source language differ from the total transfer-expression contract used here.

The finite-ground congruence method underlying K1 is inherited from Paper II, building on the initial-model organization of Paper I [1, 2]. The comparable-pair principal-congruence formula for semilattices used in Section 4 is attributed to Martin-Maroto, Ricciardo, and de Polavieja [8]. This paper’s contribution is the stated relation-matrix and sparse-representation contracts, together with the finite KnownBits classification and the proved composition of source reconstruction, local replay, and all-width target checking. The supplied evidence ties those claims to named implementations without identifying a tested program with a machine-checked proof.

11 Conclusion

Initial semantics identifies the equalities forced by observations; equality visibility explains their finite proof interfaces. The present paper turns that line into explicit kernel contracts and a versioned account of the public tool. Full relation matrices compute the finite interface exactly, and sparse implementations have a concrete coverage and closure obligation. For the released KnownBits checker, sound local rewrites connect source text to a finite semantic quotient, where nine rows decide the supported target property at every positive width.

The distinction between those layers is operationally useful. It tells users what a successful certificate means and tells maintainers what must be proved when the algorithm changes. Future revisions can enlarge the supported core while retaining the release-specific evidence and citation history that make the software’s scientific claims inspectable.

A Semantic operations and the complete rule catalogue

A.1 Word operations used by the frontend

Let $M = 2^w - 1$. Integer literals are reduced modulo 2^w , and signed values use two’s-complement interpretation. Bitwise NOT is $M \text{ XOR } x$. Addition, subtraction, and multiplication are modular. Unsigned and signed minima/maxima use their respective orders.

For unsigned shift count c , logical shifts return zero if $c \geq w$; arithmetic right shift fills with the sign bit, including for $c \geq w$. Clearing or setting c low/high bits affects $\min(c, w)$ positions. Sign-bit operations affect position $w - 1$. Leading/trailing zero/one counts are the number of consecutive matching bits from the chosen end, in $[0, w]$.

Unsigned division/remainder by zero return M and the dividend, respectively. Signed division truncates toward zero and is then reduced modulo width; division by zero returns one for a negative dividend and M otherwise. Signed remainder has the dividend’s sign and returns the dividend at divisor zero. Comparisons `cmp0` through `cmp9` denote, in order, equality, inequality, signed $<$, $\leq$, $>$, $\geq$, and unsigned $<$, $\leq$, $>$, $\geq$. Boolean connectives operate on Boolean values; selection chooses a word branch by a Boolean condition. Pair construction and projection have their ordinary meanings.

These definitions specify a pure total word language. They are not a claim that an arbitrary LLVM/MLIR backend implements this contract. The exact contract identifier and source grammar are part of the release manifest.

A.2 All 29 identifiers

In the table, 0 and $\top$ denote zero and all-ones words, W denotes the width word, and 1 denotes the ordinary integer-one word. An identity with two operand orders is used only when the implementation permits both. Guards are those of Lemma 6.2.

Identifier	Accepted law or condition
<code>idempotent</code>	$x \circ x = x$ for bitwise AND/OR and signed/unsigned min/max.
<code>self-cancel</code>	$x \text{ XOR } x = 0$ and $x - x = 0$.
<code>zero-and</code>	AND or multiplication with either operand zero gives zero.
<code>ones-or</code>	OR with either operand all-ones gives all-ones.
<code>zero-identity</code>	OR, XOR, and addition with zero on either side; subtraction with zero on the right.
<code>ones-and</code>	AND with all-ones on either side returns the other operand.
<code>not-constant</code>	NOT interchanges zero and all-ones.
<code>not-not</code>	Double bitwise NOT is identity.
<code>disjoint-and</code>	$x \& y = 0$ when the disjoint-support guard succeeds.
<code>disjoint-add</code>	$x + y = x \mid y$ under the same guard.
<code>clear-width</code>	Clearing W low or high bits gives zero.
<code>count-constant</code>	A matching bit count on a constant word is W ; a nonmatching count is zero.
<code>clear-zero-count</code>	Clearing zero low/high bits is identity; clearing bits from zero gives zero.
<code>set-zero-count</code>	Setting zero low/high bits is identity.
<code>set-leading-included</code>	Setting <code>countl_one(x)</code> high bits of y is identity when support of x is included in support of y .
<code>umin-zero</code>	Unsigned minimum with zero gives zero.
<code>umax-ones</code>	Unsigned maximum with all-ones gives all-ones.
<code>remainder-zero</code>	Unsigned or signed remainder of zero gives zero, including a zero divisor.
<code>urem-one</code>	Unsigned remainder by integer one gives zero.
<code>udiv-one</code>	Unsigned division by integer one is identity.
<code>count-bound</code>	$W <_u \text{count}(x)$ is false and $W \geq_u \text{count}(x)$ is true.
<code>compare-reflexive</code>	Reflexive comparison is true for eq, signed/unsigned $\leq, \geq$; false for ne and strict comparisons.
<code>compare-unsigned-bound</code>	Strict comparisons beyond unsigned endpoints are false; inclusive comparisons from zero or to all-ones are true. Also $0 <_u x$ and $x >_u 0$ are true if the low-bit guard proves one.
<code>select-constant</code>	A constant Boolean condition selects its corresponding branch.
<code>select-same</code>	Equal branches remove a selection.
<code>bool-constant</code>	Evaluate two Boolean constants; false absorbs Boolean AND and true absorbs Boolean OR.
<code>set-low-one</code>	Setting one low bit of zero returns integer one.

Identifier	Accepted law or condition
<code>zero-shift</code>	Shifting zero in any supported direction returns zero.
<code>shift-zero-count</code>	Shifting by zero is identity.

A.3 Proof arguments by family

Bitwise identities follow separately at every bit; modular cancellation and zero multiplication follow in $\mathbb{Z}/2^w\mathbb{Z}$. Signed and unsigned min/max are idempotent because each is a minimum or maximum in a total order. No signed-order law is substituted for an unsigned-order law.

Disjoint addition is ordinary addition without carries: at each position at most one operand bit is one, and induction from carry zero keeps carry zero. Reduction modulo 2^w then gives OR. In the leading-bit rule, every leading one of x is already a one of y by support inclusion; setting these positions changes no value. These arguments use proved guards rather than sampled word instances.

The clear/set/count rules follow directly from the affected position sets and the definitions of counts. Every count is at most w , and the encoded width word is its exact unsigned integer value. The integer-one rules remain valid at width one. That does not make arbitrary uses of integer one coordinatewise.

Unsigned comparisons with zero and M use the extrema of B_w . An operand with low bit one is nonzero, hence strictly positive in unsigned order. Reflexive comparisons and Boolean/selection identities follow from their truth tables. Remainder of zero is zero for nonzero divisors and, by the specified total extension, also for zero divisors. Division or unsigned remainder by one has the indicated value. The shift identities also cover oversized counts because the shifted zero’s sign is zero.

The rule catalogue specifies local semantic equalities. Checking that the implementation tests the matching syntax and side conditions is part of the code audit and release regressions. An altered rule set requires a renewed correspondence check before applying the old source-to-verdict theorem to a new executable.

References

- [1] L. Shcherbakov. *Initial semantics of incompletely specified actions: forced quotients, kernel saturation, and completion*. QKF project, Paper I, preprint v1.0, manuscript revision 11 September 2026. Companion manuscript; no DOI assigned in this package.
- [2] L. Shcherbakov. *Equality-visibility depth in equational presentations: finite-ground spectra, certificates, and semigroup space*. QKF project, Paper II, preprint v1.0, manuscript revision 12 September 2026. Companion manuscript; no DOI assigned in this package.
- [3] L. Shcherbakov. *QKF Certifier: Width-independent KnownBits certificates*. Software v0.1.0a1, 2026. MIT license. <https://github.com/Len989/QKF-Certifier/tree/f93561682319e8b911ed32c01583f2a496dc59fc>.
- [4] X. Peng, D. Kennedy, Y. Fan, B. Greenman, J. Regehr, and L. D’Antoni. *Nice to Meet You: Synthesizing Practical MLIR Abstract Transformers*. POPL, 2026. <https://doi.org/10.1145/3776722>. Preprint: <https://arxiv.org/abs/2512.06442>.
- [5] X. Peng et al. *Artifact for Nice to Meet You: Synthesizing Practical Abstract Transformers for MLIR*. Version v2 (Updated README), 2025. <https://doi.org/10.5281/zenodo.17371668>. Example provenance: <https://github.com/Hatsunespica/xdsl-smt/tree/4ec509edab067d62c00f76778789170061ae299c>.

- [6] LLVM Project. *llvm::KnownBits structure reference*. Official documentation, accessed 12 September 2026. https://llvm.org/doxygen/structllvm_1_1KnownBits.html.
- [7] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte. *Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers*. CGO, 2022. <https://doi.org/10.1109/CGO53902.2022.9741267>. <https://arxiv.org/abs/2105.05398>.
- [8] F. Martin-Maroto, A. Ricciardo, and G. G. de Polavieja. *The pairwise distributive law of semilattice congruences*. Algebra Universalis 87, article 29, 2026. <https://doi.org/10.1007/s00012-026-00942-x>.
- [9] A. Niemetz, M. Preiner, A. Reynolds, Y. Zohar, C. Barrett, and C. Tinelli. *Towards Bit-Width-Independent Proofs in SMT Solvers*. CADE, 2019. <https://arxiv.org/abs/1905.10434>.
- [10] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. *Alive2: Bounded Translation Validation for LLVM*. PLDI, 2021. <https://doi.org/10.1145/3453483.3454030>.